

Crisis Monitor v1.0

Sistema de detección de crisis en redes sociales mediante análisis de sentimiento en español. Clasifica comentarios en cinco categorías, genera alertas configurables y persiste los resultados en TXT, CSV, Pickle y una base de datos **MySQL**.

Plataforma: Python 3.8+ — biblioteca estándar + `mysql-connector-python` (único requisito externo, usado solo en la persistencia).

Base de datos: MySQL (servidor local). El sistema crea la base `crisis_monitor` y sus tablas automáticamente al iniciar.

Interfaz: menú interactivo de consola.

Clasificación: reglas léxicas con diccionario ponderado en español (375 entradas, incluye jerga peruana e intensificadores) — sin modelos preentrenados de IA.

Requisitos previos

Antes del primer arranque, en la máquina donde se ejecuta el sistema:

```
pip install mysql-connector-python
```

1. Tener un **servidor MySQL en ejecución** (local, en `localhost:3306`).
2. Configurar usuario y contraseña del servidor en `CONFIG_MYSQL` dentro de `persistencia.py` (por defecto `user="root"` , `password=""`).

Inicio rápido

```
python main.py
```

Dos caminos:

- **Demo express:** elige la opción `9` para generar un CSV de ejemplo y luego `1` para analizarlo. Tienes tu primer reporte de crisis en menos de 2 minutos.
- **Dataset del proyecto:** elige la opción `1` y escribe `comentarios_x.csv` (672 comentarios en español que simulan publicaciones de la plataforma X).

Dataset del proyecto

El sistema acepta cualquier CSV con las columnas `id` , `texto` , `fecha` . La columna `usuario` es **opcional**: si está presente, el sistema la conserva y la almacena (base de datos, CSV de exportación y alertas), aunque no influye en la puntuación de sentimiento.

El dataset oficial es `comentarios_x.csv` : 672 comentarios en español que simulan publicaciones de X, contruidos a partir de casos públicos documentados en redes sociales peruanas. Tras la limpieza quedan 657 comentarios únicos (13 duplicados y 2 entradas de ruido descartadas). Incluye deliberadamente filas vacías, duplicados y textos con ruido (símbolos, enlaces, jerga) para demostrar el proceso de limpieza y normalización.

La recolección automática desde X queda fuera del alcance (la API oficial eliminó su tier gratuito en 2023 y el scraping no autorizado infringe los términos de servicio de la plataforma). Ver *Arquitectura* para la decisión completa.

Módulos

Módulo	Responsabilidad
<code>main.py</code>	Punto de entrada; menú interactivo de 9 opciones
<code>ingesta.py</code>	Leer CSV, limpiar texto, detectar duplicados y filtrar ruido
<code>clasificador.py</code>	Análisis de sentimiento con léxico ponderado en español
<code>alertas.py</code>	Alertas individuales y alerta de crisis global
<code>persistencia.py</code>	Guardar y leer en TXT, CSV, Pickle y MySQL (DML completo)
<code>reportes.py</code>	Reporte de crisis en texto y resumen en consola
<code>limpiar.py</code>	Utilidad: convierte texto crudo (<code>raw.txt</code>) en <code>comentarios_x.csv</code>

Documentación

Documento	Tipo	Descripción
Tutorial: tu primer análisis	Tutorial	Desde cero hasta tu primer reporte de crisis
Cómo hacer tareas comunes	How-to	Preparar MySQL, cargar CSV, configurar umbrales, exportar datos
Referencia de módulos	Referencia	Parámetros, tipos, retornos, ejemplos
Arquitectura del sistema	Explicación	Por qué el sistema funciona como funciona

Flujo de datos

CSV de entrada → ingesta → clasificador → alertas → persistencia → reportes

Cada etapa tiene responsabilidad única. Ver *Arquitectura* para el diagrama completo y las decisiones de diseño.

Salidas generadas

Después de cargar un CSV con la opción **1**:

```
datos_salida/  
├ comentarios.txt    ← todos los comentarios con categoría y puntuación  
├ negativos.csv      ← solo comentarios negativos/muy negativos  
└ datos.pkl          ← backup completo (datos + alertas)  
  
Base de datos MySQL: crisis_monitor ← historial completo en el servidor MySQL  
                        (no es un archivo; vive en el servidor – tablas comentarios y alertas)  
  
datos_salida/reporte_crisis.txt      ← reporte de crisis (generado con opción 5)
```

La base de datos se gestiona con los cuatro verbos DML: **INSERT, SELECT, UPDATE y DELETE** (demo en vivo con la opción 7 del menú).

Umbrales por defecto

Parámetro	Valor	Configurable
Umbral de alerta individual	-2.0	Sí (opción 8)
% de negativos para crisis global	30%	No (en <code>main.py</code>)
Límite de crisis ALTO	30%	No (en <code>reportes.py</code>)
Límite de crisis CRÍTICO	60%	No (en <code>reportes.py</code>)